Concept Note: Preprocessing and Encoding Different Data Types for Machine Learning (With Examples)

Abstract

Machine learning algorithms fundamentally operate on numerical data, yet real-world datasets contain diverse data types including categorical variables, text, images, time series, and more. This concept note outlines systematic approaches to preprocess and encode different data types with practical examples to illustrate each technique.

1 Introduction

The quality of machine learning models depends heavily on how well input data is prepared. Raw data often contains inconsistencies, missing values, and non-numerical formats that algorithms cannot process directly. This guide provides concrete examples of transforming disparate data types into numerical representations.

2 Numerical Data

2.1 Continuous Variables

Example Dataset: House Prices

House_ID	Price ($)	Area (sqft)	Age (years)
1	250,000	1,500	10
2	180,000	1,200	25
3	420,000	2,400	5
4	195,000	1,350	18

2.1.1 Standardization (Z-score normalization)

Formula: $z = (x - \mu)/\sigma$

Before: Area: [1500, 1200, 2400, 1350]

After Standardization: Mean = 1612.5, Std = 492.3. Standardized Area: [-0.23, -0.84, 1.60, -0.53]

```python
from sklearn.preprocessing import StandardScaler

area = [[1500], [1200], [2400], [1350]]
scaler = StandardScaler()
scaled_area = scaler.fit_transform(area)
# Result: [[-0.23], [-0.84], [1.60], [-0.53]]
```

2.1.2 Min-Max Scaling

Formula: $x_{\text{scaled}} = (x - x_{\text{min}})/(x_{\text{max}} - x_{\text{min}})$

Before: Age: [10, 25, 5, 18]

After Min-Max Scaling (to [0,1]): Scaled Age: [0.25, 1.00, 0.00, 0.65]

```python
from sklearn.preprocessing import MinMaxScaler

age = [[10], [25], [5], [18]]
scaler = MinMaxScaler()
scaled_age = scaler.fit_transform(age)
# Result: [[0.25], [1.00], [0.00], [0.65]]
```

2.1.3 Log Transformation

Use Case: Right-skewed income data

Before: Income: [30000, 45000, 120000, 850000]

After Log Transform: Log Income: [10.31, 10.71, 11.70, 13.65]

```python
import numpy as np

income = np.array([30000, 45000, 120000, 850000])
log_income = np.log(income)
# Result: [10.31, 10.71, 11.70, 13.65]
```

3 Categorical Data

3.1 Nominal Categories

Example Dataset: Customer Data

Customer_ID	Color Preference	Country	Purchase_Amount
1	Red	USA	150
2	Blue	Canada	200
3	Green	USA	175
4	Red	Mexico	125
5	Blue	Canada	220

3.1.1 One-Hot Encoding

Before: Color: [Red, Blue, Green, Red, Blue]

After One-Hot Encoding:

Color_Red	Color_Blue	Color_Green
1	0	0
0	1	0
0	0	1
1	0	0
0	1	0

```
import pandas as pd

df = pd.DataFrame({'Color': ['Red', 'Blue', 'Green', 'Red', 'Blue']})
one_hot = pd.get_dummies(df['Color'], prefix='Color')
```

3.1.2 Target Encoding

Before:

Country	Purchase_Amount
USA	150
Canada	200
USA	175
Mexico	125
Canada	220

Calculate Mean Purchase by Country:

- USA: (150 + 175) / 2 = 162.5
- Canada: (200 + 220) / 2 = 210.0
- Mexico: 125 / 1 = 125.0

After Target Encoding:

Country_Encoded
162.5
210.0
162.5
125.0
210.0

```
import pandas as pd

df = pd.DataFrame({
    'Country': ['USA', 'Canada', 'USA', 'Mexico', 'Canada'],
    'Purchase_Amount': [150, 200, 175, 125, 220]
})

target_means = df.groupby('Country')['Purchase_Amount'].mean()
df['Country_Encoded'] = df['Country'].map(target_means)
```

3.1.3 Frequency Encoding

Before: Product_ID: [A123, B456, A123, C789, A123, B456, D012]

Calculate Frequencies:

- A123: 3 occurrences
- B456: 2 occurrences
- C789: 1 occurrence
- D012: 1 occurrence

After Frequency Encoding: Product_Frequency: [3, 2, 3, 1, 3, 2, 1]

3.2 Ordinal Categories

Example: Education Level

Before: Education: [High School, Bachelor, Master, Bachelor, PhD, High School]

After Label Encoding:

Education	Encoded
High School	1
Bachelor	2
Master	3
PhD	4

Result: Education_Encoded: [1, 2, 3, 2, 4, 1]

```
from sklearn.preprocessing import LabelEncoder

education_mapping = {
    'High School': 1,
    'Bachelor': 2,
    'Master': 3,
    'PhD': 4
}

education = ['High School', 'Bachelor', 'Master', 'Bachelor', 'PhD', 'High School']
encoded = [education_mapping[e] for e in education]
# Result: [1, 2, 3, 2, 4, 1]
```

4 Text Data

4.1 Traditional Approaches

Example: Product Reviews

- Review 1: ”This product is amazing! Best purchase ever.”
- Review 2: ”Terrible quality. Would not recommend.”
- Review 3: ”Good product, fast shipping.”

4.1.1 Preprocessing Steps

Step 1: Lowercasing

- Review 1: ”this product is amazing! best purchase ever.”
- Review 2: ”terrible quality. would not recommend.”
- Review 3: ”good product, fast shipping.”

Step 2: Tokenization

- Review 1: [’this’, ’product’, ’is’, ’amazing’, ’best’, ’purchase’, ’ever’]
- Review 2: [’terrible’, ’quality’, ’would’, ’not’, ’recommend’]
- Review 3: [’good’, ’product’, ’fast’, ’shipping’]

Step 3: Remove Stop Words

- Review 1: [’product’, ’amazing’, ’best’, ’purchase’, ’ever’]
- Review 2: [’terrible’, ’quality’, ’recommend’]
- Review 3: [’good’, ’product’, ’fast’, ’shipping’]

4.1.2 Bag of Words (BoW)

Vocabulary: [amazing, best, ever, fast, good, product, purchase, quality, recommend, shipping, terrible]

BoW Matrix:

amaz	best	ever	fast	good	prod	purch	qual	recom	ship	terr
1	1	1	0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	1	1	0	1
0	0	0	1	1	1	0	0	0	1	0

```
from sklearn.feature_extraction.text import CountVectorizer

reviews = [
    "This product is amazing! Best purchase ever.",
    "Terrible quality. Would not recommend.",
    "Good product, fast shipping."
]

vectorizer = CountVectorizer()
bow_matrix = vectorizer.fit_transform(reviews)
print(vectorizer.get_feature_names_out())
print(bow_matrix.toarray())
```

4.1.3 TF-IDF Encoding

Formula: $\text{TF-IDF} = \text{TF} \times \text{IDF} = (\text{term frequency}) \times \log(N/\text{document frequency})$

Example Calculation for "product":

- Appears in Review 1: 1 time
- Appears in Review 3: 1 time
- Total documents: 3
- Document frequency: 2
- TF (Review 1) = 1/7 = 0.143
- IDF = log(3/2) = 0.176
- TF-IDF (Review 1) = 0.143 $\times$ 0.176 = 0.025

```
from sklearn.feature_extraction.text import TfidfVectorizer

reviews = [
    "This product is amazing! Best purchase ever.",
    "Terrible quality. Would not recommend.",
    "Good product, fast shipping."
]

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(reviews)
```

4.2 N-grams

Bigrams (2-grams) Example

- Text: ”fast shipping good product”
- Unigrams: [’fast’, ’shipping’, ’good’, ’product’]
- Bigrams: [’fast shipping’, ’shipping good’, ’good product’]

```
from sklearn.feature_extraction.text import CountVectorizer

text = ["fast shipping good product"]
vectorizer = CountVectorizer(ngram_range=(1, 2))
ngrams = vectorizer.fit_transform(text)
print(vectorizer.get_feature_names_out())
# Output: ['fast', 'fast shipping', 'good', 'good product',
#          'product', 'shipping', 'shipping good']
```

5 Date and Time Data

Example Dataset: E-commerce Transactions

Transaction_ID	Timestamp	Amount
1	2024-03-15 14:30:00	150
2	2024-03-15 09:15:00	200
3	2024-03-16 18:45:00	175
4	2024-03-17 12:00:00	125

5.1 Feature Extraction

Extracted Features:

Year	Month	Day	Hour	DayOfWeek	IsWeekend	Quarter
2024	3	15	14	4 (Fri)	0	1
2024	3	15	9	4 (Fri)	0	1
2024	3	16	18	5 (Sat)	1	1
2024	3	17	12	6 (Sun)	1	1

```python
import pandas as pd

df = pd.DataFrame({
    'Timestamp': pd.to_datetime([
        '2024-03-15 14:30:00',
        '2024-03-15 09:15:00',
        '2024-03-16 18:45:00',
        '2024-03-17 12:00:00'
    ])
})

df['Year'] = df['Timestamp'].dt.year
df['Month'] = df['Timestamp'].dt.month
df['Day'] = df['Timestamp'].dt.day
df['Hour'] = df['Timestamp'].dt.hour
df['DayOfWeek'] = df['Timestamp'].dt.dayofweek
df['IsWeekend'] = df['DayOfWeek'].isin([5, 6]).astype(int)
df['Quarter'] = df['Timestamp'].dt.quarter
```

5.2 Cyclical Encoding

Problem: Hour 23 and Hour 0 are adjacent but numerically distant

Solution: Use sine and cosine transformations

For Hour = 14:

- $\text{Hour}_{\text{sin}} = \sin(2\pi \times 14/24) = \sin(3.665) = -0.259$
- $\text{Hour}_{\text{cos}} = \cos(2\pi \times 14/24) = \cos(3.665) = 0.966$

```python
import numpy as np
import pandas as pd

df = pd.DataFrame({'Hour': [14, 9, 18, 12]})

df['Hour_sin'] = np.sin(2 * np.pi * df['Hour'] / 24)
df['Hour_cos'] = np.cos(2 * np.pi * df['Hour'] / 24)

# Result:
# Hour_sin: [-0.259, 0.707, -0.866, 0.000]
# Hour_cos: [0.966, 0.707, 0.500, 1.000]
```

6 Image Data

Example: Grayscale Image (28×28 pixels)

Original Image: 28×28 pixel matrix with values 0-255

Pixel Matrix (simplified 4×4):

$$\begin{bmatrix} 0 & 45 & 120 & 200 \\ 30 & 80 & 150 & 210 \\ 10 & 60 & 140 & 195 \\ 5 & 50 & 130 & 180 \end{bmatrix}$$

6.1 Normalization

Scale to [0, 1]: Normalized $=$ Original$/255$

$$\begin{bmatrix} 0.000 & 0.176 & 0.471 & 0.784 \\ 0.118 & 0.314 & 0.588 & 0.824 \\ 0.039 & 0.235 & 0.549 & 0.765 \\ 0.020 & 0.196 & 0.510 & 0.706 \end{bmatrix}$$

```
import numpy as np

image = np.array([
    [0, 45, 120, 200],
    [30, 80, 150, 210],
    [10, 60, 140, 195],
    [5, 50, 130, 180]
])

normalized_image = image / 255.0
```

6.2 Flattening for ML Models

From 28×28 Matrix to 784-length Vector:

- Original shape: (28, 28)
- Flattened shape: (784,)

```
import numpy as np

image = np.random.randint(0, 256, (28, 28))
flattened = image.flatten()
print(flattened.shape)  # Output: (784,)
```

7 Missing Data

Example Dataset: Patient Records

Patient_ID	Age	Blood_Pressure	Cholesterol	Outcome
1	45	120	NaN	0
2	NaN	130	200	1
3	52	NaN	180	0
4	38	125	195	1
5	61	140	NaN	0

7.1 Mean Imputation

Calculate Means:

- Age mean: (45 + 52 + 38 + 61) / 4 = 49
- Blood_Pressure mean: (120 + 130 + 125 + 140) / 4 = 128.75
- Cholesterol mean: (200 + 180 + 195) / 3 = 191.67

After Imputation:

Patient_ID	Age	Blood_Pressure	Cholesterol	Outcome
1	45	120	191.67	0
2	49	130	200	1
3	52	128.75	180	0
4	38	125	195	1
5	61	140	191.67	0

```
import pandas as pd
from sklearn.impute import SimpleImputer

df = pd.DataFrame({
    'Age': [45, None, 52, 38, 61],
    'Blood_Pressure': [120, 130, None, 125, 140],
    'Cholesterol': [None, 200, 180, 195, None]
})

imputer = SimpleImputer(strategy='mean')
df_imputed = pd.DataFrame(
    imputer.fit_transform(df),
    columns=df.columns
)
```

7.2 Missing Indicator

Add Binary Flags:

Age	BP	Chol	Age_Missing	BP_Missing	Chol_Missing
45	120	191.67	0	0	1
49	130	200	1	0	0
52	129	180	0	1	0
38	125	195	0	0	0
61	140	191.67	0	0	1

```
from sklearn.impute import MissingIndicator
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Age': [45, np.nan, 52, 38, 61],
    'Cholesterol': [np.nan, 200, 180, 195, np.nan]
})

indicator = MissingIndicator()
missing_flags = indicator.fit_transform(df)
```

8 High-Cardinality Categorical Features

Example: User IDs (1000+ unique values)

Original Data: User_ID: [U_19283, U_74628, U_19283, U_92847, U_74628, ...]

8.1 Grouping Rare Categories

Frequency Count:

- U_19283: 150 occurrences
- U_74628: 120 occurrences
- U_92847: 5 occurrences
- U_10293: 3 occurrences

After Grouping (threshold=10):

Original	Grouped
U_19283	U_19283
U_74628	U_74628
U_92847	Other
U_10293	Other

```
import pandas as pd

df = pd.DataFrame({
    'User_ID': ['U_19283', 'U_74628', 'U_19283', 'U_92847', 'U_10293']
})

# Count frequencies
value_counts = df['User_ID'].value_counts()

# Map rare categories to 'Other'
threshold = 10
df['User_ID_Grouped'] = df['User_ID'].apply(
    lambda x: x if value_counts[x] >= threshold else 'Other'
)
```

8.2 Embedding Approach

Convert High-Cardinality IDs to Dense Vectors:

- Original: U_19283 → [0.23, -0.45, 0.67, 0.12, -0.89]
- U_74628 → [-0.34, 0.56, -0.12, 0.78, 0.23]

(5-dimensional embedding)

```
from tensorflow.keras.layers import Embedding
from tensorflow.keras.models import Sequential
import numpy as np

# Assume 1000 unique users, create 8-dimensional embeddings
num_users = 1000
embedding_dim = 8

model = Sequential([
    Embedding(input_dim=num_users, output_dim=embedding_dim)
])

# User IDs [0, 1, 2]
user_ids = np.array([0, 1, 2])
embeddings = model.predict(user_ids)
# Output shape: (3, 8) - three users with 8-dim vectors each
```

9 Imbalanced Data

Example: Fraud Detection Dataset

Transaction_ID	Amount	Is_Fraud
1	150	0
2	200	0
3	175	0
4	125	1
5	220	0
6	300	0
...	...	...

Class Distribution:

- Non-Fraud (0): 9,850 samples (98.5%)
- Fraud (1): 150 samples (1.5%)

9.1 SMOTE (Synthetic Minority Over-sampling)

Before SMOTE: Fraud samples: 150

After SMOTE: Fraud samples: 9,850 (synthetic samples created)

Example Synthetic Sample: Take two fraud samples:

- Sample A: [150, 25, 1.2]
- Sample B: [175, 30, 1.5]

New synthetic sample = A + random(0,1) $\times$ (B - A) = [150, 25, 1.2] + 0.6 $\times$ [25, 5, 0.3] = [165, 28, 1.38]

```
from imblearn.over_sampling import SMOTE
import numpy as np

X = np.array([[150, 25], [200, 30], [175, 28],
              [125, 22], [220, 32], [300, 35]])
y = np.array([0, 0, 0, 1, 0, 0])

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

print(f"Original shape: {X.shape}")
print(f"Resampled shape: {X_resampled.shape}")
print(f"Original class distribution: {np.bincount(y)}")
print(f"Resampled class distribution: {np.bincount(y_resampled)}")
```

10 Complete Pipeline Example

Real-World Dataset: Customer Churn Prediction

Raw Data:

CustomerID	Age	Gender	Tenure	MonthlyCharges	Contract	Churn
C001	29	Male	12	65.50	Month-to-month	No
C002	NaN	Female	24	89.99	One year	No
C003	45	Male	6	45.00	Month-to-month	Yes
C004	52	Female	36	110.50	Two year	No

(Note: LastContact column omitted for brevity in table)

10.1 Step-by-Step Preprocessing

1. **Handle Missing Values:**

```
# Impute Age with median
df['Age'].fillna(df['Age'].median(), inplace=True)

# Impute LastContact with forward fill
df['LastContact'].fillna(method='ffill', inplace=True)
```

2. **Encode Gender (Binary):**

```
df['Gender_Encoded'] = df['Gender'].map({'Male': 0, 'Female': 1})
```

3. **One-Hot Encode Contract:**

```
contract_dummies = pd.get_dummies(df['Contract'], prefix='Contract')
# Creates: Contract_Month-to-month, Contract_One year, ...
```

4. **Extract Date Features:**

```
df['Contact_Hour'] = pd.to_datetime(df['LastContact']).dt.hour
df['Contact_DayOfWeek'] = pd.to_datetime(df['LastContact']).dt.dayofweek
```

5. **Scale Numerical Features:**

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[['Age', 'Tenure', 'MonthlyCharges']] = scaler.fit_transform(
    df[['Age', 'Tenure', 'MonthlyCharges']]
)
\end{falsestlisting}
    \item \textbf{Encode Target Variable:}
\begin{lstlisting}
df['Churn_Encoded'] = df['Churn'].map({'No': 0, 'Yes': 1})
```

Final Processed Data (Example):

Age_S	Tenure_S	Charges_S	Gender_E	Cont_MTM	Cont_1Y	Cont_2Y	Hour	Churn
-0.85	-0.45	-0.23	0	1	0	0	14	0
0.42	0.78	0.56	1	0	1	0	9	0
0.23	-1.12	-1.45	0	1	0	0	18	1
1.05	1.89	1.78	1	0	0	1	14	0

11 Best Practices Summary

1. **Always Split Data First:**

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

# Fit preprocessing ONLY on training data
scaler.fit(X_train)

# Transform both sets
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

2. **Use Pipelines:**

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier())
])

pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```

3. **Document Your Transformations:**

```
preprocessing_steps = {
    'missing_values': 'Mean imputation for Age, ffill for LastContact',
    'encoding': 'One-hot for Contract, Label encoding for Gender',
    'scaling': 'StandardScaler for Age, Tenure, MonthlyCharges',
    'date_features': 'Extracted Hour and DayOfWeek from LastContact'
}
```

12 Conclusion

Effective preprocessing and encoding require understanding both the data types and the algorithms being used. The examples provided demonstrate that the same data can be encoded in multiple ways, and the choice depends on:

- The nature of the data (nominal vs ordinal, sparse vs dense)
- The algorithm requirements (tree-based vs linear models)
- The size of the dataset (small datasets may not support complex encodings)
- The domain context (whether certain transformations preserve meaningful relationships)

By systematically applying these techniques with proper validation and pipeline management, practitioners can transform raw, heterogeneous data into clean, numerical representations that enable machine learning models to learn effectively and generalize well to new data.